

Algorithm Analysis - Practice Questions

With Answers

SECTION A: MULTIPLE CHOICE QUESTIONS

****Question 1:**** What does Big-O notation describe?

- A) The exact number of operations an algorithm performs
- B) The upper bound or worst-case growth rate of an algorithm
- C) The best-case performance of an algorithm
- D) The amount of memory an algorithm uses

****Answer: B****

***Explanation:** Big-O notation describes the upper bound - how fast an algorithm grows in the worst case. It's not exact (not A), not best case (not C), and not about memory (not D).*

****Question 2:**** In linear search, which scenario requires checking ALL elements in an array of size n ?

- A) Best case
- B) Average case
- C) Worst case
- D) Every case

****Answer: C****

***Explanation:** Worst case occurs when the target element is at the last position or not in the array at all, requiring all n comparisons.*

****Question 3:**** What is the time complexity of this code snippet?

```
```c
for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
 printf("Hello");
```
```

- A) $O(n)$
- B) $O(\log n)$
- C) $O(n^2)$
- D) $O(2^n)$

****Answer: C****

***Explanation:** Nested loops where both run n times give $n \times n = n^2$ operations.*

****Question 4:**** Which complexity class is the FASTEST?

- A) $O(n^2)$
- B) $O(n \log n)$
- C) $O(\log n)$
- D) $O(2^n)$

****Answer: C****

***Explanation:** $O(\log n)$ grows much slower than $O(n)$, $O(n \log n)$, $O(n^2)$, and $O(2^n)$. For $n=1000$, $O(\log n) \approx 10$ operations while $O(n) = 1000$.*

****Question 5:**** If an algorithm takes $3n^2 + 5n + 2$ operations, what is its Big-O complexity?

- A) $O(3n^2)$
- B) $O(n^2 + n)$
- C) $O(n^2)$
- D) $O(n)$

****Answer: C****

Explanation: Drop constants (3) and lower-order terms ($5n + 2$). The dominant term is n^2 , so $O(n^2)$.

****Question 6:**** What does Ω (Big-Omega) notation represent?

- A) Upper bound
- B) Lower bound
- C) Tight bound
- D) Average bound

****Answer: B****

Explanation: Big-Omega gives a lower bound - a guarantee that the algorithm takes AT LEAST this many operations.

****Question 7:**** For bubble sort on an already sorted array, what is the best-case time complexity?

- A) $O(n^2)$
- B) $O(n \log n)$
- C) $O(n)$
- D) $O(1)$

****Answer: C****

***Explanation:** With optimization (checking if swaps occurred), bubble sort can detect a sorted array in one pass: $O(n)$.*

****Question 8:**** Which of the following is TRUE about worst-case analysis?

- A) It's rarely used in practice
- B) It provides an overly optimistic view
- C) It's essential for real-time and safety-critical systems
- D) It's harder to calculate than average case

****Answer: C****

***Explanation:** Worst-case analysis dominates in practice because it provides guarantees needed for reliability, especially in medical devices, flight control, etc.*

****Question 9:**** What is the time complexity of binary search?

- A) $O(n)$
- B) $O(n^2)$
- C) $O(\log n)$
- D) $O(1)$

****Answer: C****

***Explanation:** Binary search repeatedly divides the search space in half, giving logarithmic time $O(\log n)$.*

****Question 10:**** If an algorithm has time complexity $O(2^n)$, what happens when n increases from 10 to 20?

- A) Time roughly doubles
- B) Time roughly increases by a factor of 1000
- C) Time roughly increases by a factor of 1,000,000
- D) Time roughly stays the same

****Answer: C****

***Explanation:** $2^{10} = 1024$, $2^{20} = 1,048,576$. That's about a $1000\times$ increase!*

****Question 11:**** Which notation indicates that an algorithm has the SAME asymptotic upper and lower bound?

- A) Big-O
- B) Big-Omega
- C) Big-Theta

D) Little-o

****Answer: C****

Explanation: Big-Theta (Θ) means $f(n) = O(g(n))$ AND $f(n) = \Omega(g(n))$ - the bounds match tightly.

****Question 12:**** What is the time complexity of this code?

```
```c
```

```
for (int i = 0; i < n; i++)
```

```
 doSomething(); // O(1) operation
```

```
for (int i = 0; i < n; i++)
```

```
 for (int j = 0; j < n; j++)
```

```
 doSomethingElse(); // O(1) operation
```

```
```
```

A) $O(n)$

B) $O(n^2)$

C) $O(n + n^2)$

D) $O(n^2)$

****Answer: D****

Explanation: $O(n) + O(n^2) = O(n^2)$ because the higher-order term dominates.

****Question 13:**** For comparison-based sorting algorithms, what is the proven lower bound?

- A) $O(n)$
- B) $\Omega(n \log n)$
- C) $O(n^2)$
- D) $\Omega(n)$

****Answer: B****

***Explanation:** Information theory proves that comparison-based sorting cannot be faster than $\Omega(n \log n)$ in the worst case.*

****Question 14:**** Which of the following is $O(1)$ complexity?

- A) Finding an element in a sorted array
- B) Accessing an array element by index
- C) Sorting an array
- D) Searching in a linked list

****Answer: B****

*Explanation: Accessing `arr[i]` takes constant time regardless of array size
- it's $O(1)$.*

Question 15: What is the average case of linear search on random data?

- A) $O(1)$
- B) $O(n)$
- C) $O(n/2)$
- D) $O(\log n)$

Answer: B

Explanation: Even though it averages $n/2$ comparisons, in Big-O terms we still say $O(n)$ because constants don't matter.

Question 16: When $n = 1000$, approximately how many operations will an $O(n \log n)$ algorithm perform?

- A) 1000
- B) 10,000
- C) 100,000
- D) 1,000,000

****Answer: B****

***Explanation:** $n \log n = 1000 \times \log_2(1000) \approx 1000 \times 10 = 10,000$ operations.*

****Question 17:**** Which optimization strategy typically provides the BIGGEST performance improvement?

- A) System-level optimizations (cache locality)
- B) Removing redundant computations
- C) Choosing a better algorithm
- D) Using better data structures

****Answer: C****

***Explanation:** Changing from $O(n^2)$ to $O(n \log n)$ can give 1000x speedup. Other optimizations give smaller (2-10x) improvements.*

****Question 18:**** What does it mean when we say an algorithm is "optimal"?

- A) It runs in $O(1)$ time
- B) Its upper bound matches the problem's lower bound
- C) It uses the least memory possible
- D) It's the simplest to implement

****Answer: B****

***Explanation:** An optimal algorithm has time complexity that matches the proven lower bound for that problem - you can't do better.*

****Question 19:**** Which of these correctly simplifies $4n^3 + 100n^2 + 1000$?

- A) $O(4n^3)$
- B) $O(n^3 + n^2)$
- C) $O(n^3)$
- D) $O(1000)$

****Answer: C****

***Explanation:** Drop constants (4) and lower-order terms ($100n^2 + 1000$). The dominant term n^3 gives $O(n^3)$.*

****Question 20:**** Why is best-case analysis rarely useful in practice?

- A) It's too hard to calculate
- B) It happens infrequently and gives an overly optimistic view
- C) It requires knowledge of input distribution
- D) It's the same as average case

****Answer: B****

Explanation: Best case rarely occurs in practice and provides no guarantee about typical or worst performance.

SECTION B: TRUE OR FALSE QUESTIONS

****Question 21:**** Big-O notation tells you the exact number of seconds an algorithm will take.

****Answer: FALSE****

Explanation: Big-O describes growth rate, not exact time. It ignores constants and implementation details.

****Question 22:**** An $O(n)$ algorithm will always be faster than an $O(n^2)$ algorithm for any input size.

****Answer: FALSE****

Explanation: For very small inputs, $O(n^2)$ might be faster due to constants. For $n=5$, $O(n^2)=25$ operations vs $O(n)=5$ - but the difference is small.

****Question 23:**** Bubble sort has a best-case time complexity of $O(n)$.

****Answer: TRUE****

***Explanation:** If the array is already sorted and the algorithm checks for swaps, it can complete in one pass: $O(n)$.*

****Question 24:**** $\Omega(n)$ means the algorithm takes at most n operations.

****Answer: FALSE****

***Explanation:** $\Omega(n)$ means AT LEAST n operations - it's a lower bound, not an upper bound.*

****Question 25:**** $\Theta(n)$ means the algorithm has both upper and lower bounds of n .

****Answer: TRUE****

***Explanation:** Big-Theta means the upper bound (O) and lower bound (Ω) match - it grows exactly like n .*

****Question 26:**** Constants like the 5 in " $5n + 3$ " matter a lot in Big-O analysis.

****Answer: FALSE****

***Explanation:** Big-O drops constants because they become irrelevant for large n .*

****Question 27:**** A problem's lower bound tells us the best any algorithm can possibly do.

****Answer: TRUE****

***Explanation:** Lower bound proves that NO algorithm can solve the problem faster than this.*

****Question 28:**** $O(2^n)$ is faster than $O(n^2)$ for large values of n .

****Answer: FALSE****

***Explanation:** Exponential (2^n) grows much faster than quadratic (n^2). For $n=100$, $n^2=10,000$ but 2^n is astronomical.*

****Question 29:**** Average-case analysis requires knowing the probability distribution of inputs.

****Answer: TRUE****

***Explanation:** To calculate average case, you need to know what "typical" inputs look like.*

****Question 30:**** System-level optimizations like cache locality should be your FIRST priority when speeding up code.

****Answer: FALSE****

***Explanation:** First choose a better algorithm, THEN optimize at lower levels. Algorithm choice has the biggest impact.*

SECTION C: SHORT ANSWER QUESTIONS

****Question 31:**** Explain the difference between best case, average case, and worst case analysis. Give an example using linear search.

****Answer:****

****Best Case:**** Minimum operations with optimal input

- Linear search: Target is first element → 1 comparison

****Average Case:**** Expected operations with typical input

- Linear search: Target randomly positioned → $n/2$ comparisons on average

****Worst Case:**** Maximum operations with any input

- Linear search: Target is last or missing → n comparisons

****Key insight:**** Worst case is most useful for guarantees. Best case is too optimistic. Average case requires knowing your data distribution.

****Question 32:**** What are the three rules for simplifying expressions to Big-O notation? Apply them to: $f(n) = 7n^3 + 25n^2 + 100n + 1000$

****Answer:****

****Three rules:****

1. Drop constant multipliers (7, 25, 100, 1000)
2. Drop lower-order terms (n^2 , n , constant)
3. Keep only the dominant (highest power) term

****Applying to $f(n) = 7n^3 + 25n^2 + 100n + 1000$:**

- Drop constants: $n^3 + n^2 + n + 1$ (conceptually)
- Keep only highest power: n^3
- Result: **$O(n^3)$**

Question 33: Explain the difference between O , Ω , and Θ notations.

Answer:

| Notation | Name | Meaning | Analogy | |
|----------------------------|-----------|-----------------------------------|--------------------------------|--|
| O | Big-O | Upper bound: "No slower than" | Speed limit: ≤ 60 mph | |
| Ω | Big-Omega | Lower bound: "At least this fast" | Minimum speed: ≥ 30 mph | |
| Θ | Big-Theta | Tight bound: "Exactly this fast" | Cruise control: exactly 45 mph | |

Example with $f(n) = 3n + 2$:

- $O(n)$ ✓ (it's $\leq$ some constant $\times n$)
- $\Omega(n)$ ✓ (it's $\geq$ some constant $\times n$)
- $\Theta(n)$ ✓ (both bounds match)

Question 34: Calculate the time complexity of this nested loop:

```
```c
int count = 0;
for (int i = 1; i <= n; i++)
 for (int j = i; j <= n; j++)
 count++;
```
```

Show your work.

****Answer:****

****Step 1 - Count iterations:****

- i = 1: j runs n times
- i = 2: j runs n-1 times
- i = 3: j runs n-2 times
- ...
- i = n: j runs 1 time

****Step 2 - Sum the series:****

$$\begin{aligned}\text{Total} &= n + (n-1) + (n-2) + \dots + 1 \\ &= n(n+1)/2\end{aligned}$$

****Step 3 - Simplify:****

$$n(n+1)/2 = (n^2 + n)/2$$

****Step 4 - Apply Big-O rules:****

- Drop constant $1/2$
- Drop lower-order term n
- Result: **$O(n^2)$**

****Question 35:**** What is the difference between analyzing an algorithm and analyzing a problem? Use sorting as an example.

****Answer:****

****Analyzing an ALGORITHM:****

- Question: "How fast is THIS specific solution?"
- Example: Bubble sort = $O(n^2)$
- Example: Merge sort = $O(n \log n)$
- This tells us about a particular implementation

****Analyzing a PROBLEM:****

- Question: "How fast can ANY solution possibly be?"
- Example: Comparison-based sorting = $\Omega(n \log n)$ proven lower bound
- This tells us the inherent difficulty of the problem

****Optimal algorithm:****

When an algorithm's upper bound matches the problem's lower bound

- Merge sort = $O(n \log n)$

- Sorting lower bound = $\Omega(n \log n)$
- Therefore merge sort is ****optimal**** - no faster comparison sort exists!

****Question 36:**** List and briefly explain the four strategies for speeding up programs, in order of importance.

****Answer:****

****1. Choose a better algorithm (MOST IMPORTANT)****

- Change from $O(n^2)$ to $O(n \log n)$ - can give 1000x speedup
- Example: Use quicksort instead of bubble sort

****2. Remove redundant computation****

- Don't calculate the same thing multiple times
- Cache results for reuse
- Example: Move expensive calculations out of loops

****3. Select better data structures****

- Use hash tables for $O(1)$ search instead of $O(n)$ lists
- Use trees for range queries
- Match data structure to your operations

****4. System-level optimizations (LEAST IMPORTANT)****

- Cache locality (access memory sequentially)

- Minimize I/O operations
- These give 2-10x speedup - do them LAST

****Question 37:**** What is an optimal algorithm? Why is merge sort considered optimal for sorting?

****Answer:****

****Optimal algorithm definition:****

An algorithm is optimal when its time complexity matches the proven lower bound for that problem. You literally cannot do better asymptotically.

****Why merge sort is optimal:****

- Merge sort runs in $O(n \log n)$ time
- Computer scientists have PROVEN that any comparison-based sorting algorithm must take at least $\Omega(n \log n)$ comparisons in the worst case
- Upper bound (O) = Lower bound (Ω) = $n \log n$
- Therefore merge sort is optimal

****Important:**** This doesn't mean merge sort is always the fastest in practice (constants matter for small n), but asymptotically, no algorithm can beat it.

Question 38: Compare and contrast $O(n)$ and $O(n^2)$ algorithms. Give an example of each and explain when you might choose one over the other.

Answer:

| Aspect | $O(n)$ - Linear | $O(n^2)$ - Quadratic |
|---------------------------------|------------------------------------|-------------------------------------|
| --- | --- | --- |
| Growth | Time increases linearly with input | Time increases with square of input |
| $n=100$ | 100 operations | 10,000 operations |
| $n=1000$ | 1000 operations | 1,000,000 operations |
| $n=1,000,000$ | 1 million ops | 1 trillion ops |
| Example | Linear search, single loop | Bubble sort, nested loops |

When to choose:

- Choose $O(n)$ whenever possible - it scales much better
- Sometimes $O(n^2)$ algorithms are simpler to implement for small datasets
- For $n < 100$, $O(n^2)$ might be acceptable if constants are small
- For large n , $O(n^2)$ becomes impossible quickly

Rule of thumb: If your data might grow, prefer $O(n)$ or better!

****Question 39:**** What is the time complexity of this code? Explain your reasoning.

```
```\nc
for (int i = 0; i < n; i++) {
 for (int j = 0; j < 100; j++) {
 printf("Constant work");
 }
}
```
```

****Answer:****

****Analysis:****

- Outer loop: runs n times
- Inner loop: runs 100 times (CONSTANT, independent of n)
- Total operations = $n \times 100 = 100n$

****Time complexity: $O(n)$ ****

****Why?****

- The inner loop runs a constant number of times (100)
- Constants don't matter in Big-O
- This is linear, NOT quadratic!

****Important lesson:**** Not all nested loops are $O(n^2)$! Check if the inner loop depends on n .

****Question 40:**** Explain why worst-case analysis is preferred in safety-critical systems like medical devices or flight control software.

****Answer:****

****Why worst case matters for safety:****

****1. Guarantees, not averages****

- Medical device: "Heart monitor responds in 10ms on average" is NOT acceptable
- Must be: "Heart monitor ALWAYS responds within 10ms, even in worst case"

****2. No room for failure****

- Flight control: System slowdown during landing could cause crash
- Must budget for maximum possible time

****3. Real-time requirements****

- Many systems have hard deadlines
- Missing deadline = system failure

****4. User experience****

- Users notice lag when systems slow down
- Worst case determines perceived responsiveness

****Analogy:**** When designing a bridge, you don't plan for average weight - you plan for maximum possible load. Same with safety-critical software!

SECTION D: ESSAY QUESTIONS

****Question 41:**** You are tasked with choosing a sorting algorithm for a company's data processing system. The system will handle datasets ranging from 100 to 10 million items. Compare bubble sort ($O(n^2)$), merge sort ($O(n \log n)$), and quicksort ($O(n \log n)$ average, $O(n^2)$ worst case). Which would you recommend and why? Consider best, average, and worst cases in your answer.

****Answer:****

****Algorithm Comparison:****

| Algorithm | Best Case | Average Case | Worst Case | Space |
|-------------|---------------|---------------|---------------|--------|
| --- | --- | --- | --- | --- |
| Bubble Sort | $O(n)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ |
| Merge Sort | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$ |

| Quicksort | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$ | $O(\log n)$ |

****Analysis for different dataset sizes:****

****For $n = 100$ (small):****

- All algorithms are fast enough
- Bubble sort: 10,000 ops (fine)
- Merge sort: ~700 ops
- Quicksort: ~700 ops avg
- Any would work, but bubble sort is simplest

****For $n = 10,000$ (medium):****

- Bubble sort: 100 million ops (slow - seconds to minutes)
- Merge/quicksort: ~140,000 ops (fast - milliseconds)
- Bubble sort is eliminated

****For $n = 10$ million (large):****

- Merge/quicksort: ~230 million ops (reasonable)
- Bubble sort: 100 trillion ops (impossible - would take days/weeks)

****The Quicksort Risk:****

- Worst case $O(n^2)$ occurs on already sorted data if pivot choice is poor
- For 10 million items, worst case = 100 trillion ops (same as bubble sort!)
- This could happen if data is often pre-sorted

****Recommendation: Merge Sort****

****Reasons:****

1. ****Consistent $O(n \log n)$ **** - no bad worst case
2. ****Scales well**** to 10 million items
3. ****Predictable performance**** - important for production systems
4. ****Only downside**** - extra memory, but modern systems have plenty

****Alternative:**** Use a hybrid - introsort (quicksort that switches to heapsort if recursion gets too deep) gives quicksort's speed with guaranteed $O(n \log n)$.

****Conclusion:**** For reliability at scale, choose merge sort or a hybrid. Never use $O(n^2)$ algorithms for large datasets.

****Question 42:**** Explain the concept of asymptotic analysis. Why do computer scientists ignore constants and lower-order terms? Provide examples to illustrate your answer.

****Answer:****

****What is Asymptotic Analysis?****

Asymptotic analysis describes how algorithm performance changes as input size grows toward infinity. It focuses on growth rate, not exact operation count.

****Why Ignore Constants?****

****1. Hardware differences****

- Same algorithm: $2n$ operations
- Fast computer: 1 nanosecond per op = $2n$ nanoseconds
- Slow computer: 10 nanoseconds per op = $20n$ nanoseconds
- Both are still $O(n)$ - constant factor doesn't change the growth pattern

****2. Implementation differences****

- Python vs C: same algorithm, different constants
- Compiler optimizations: change constants
- Both still have same Big-O

****3. Constants become irrelevant for large n ****

...

$n = 1,000,000$

$O(n)$ algorithm: $1,000,000 \times C$ operations

$O(n^2)$ algorithm: 1,000,000,000,000 operations

...

No constant C can make $O(n^2)$ faster than $O(n)$ for large n

****Why Ignore Lower Terms?****

****Example:**** $f(n) = n^2 + 1000n$

| n | n^2 term | $1000n$ term | Total | % from n^2 |
|-----|------------|--------------|-------|--------------|
|-----|------------|--------------|-------|--------------|

| | | | | |
|-----|-----|-----|-----|-----|
| --- | --- | --- | --- | --- |
|-----|-----|-----|-----|-----|

| 10 | 100 | 10,000 | 10,100 | 1% |
| 100 | 10,000 | 100,000 | 110,000 | 9% |
| 1000 | 1,000,000 | 1,000,000 | 2,000,000 | 50% |
| 10,000 | 100,000,000 | 10,000,000 | 110,000,000 | 91% |
| 100,000 | 10,000,000,000 | 100,000,000 | 10,100,000,000 | 99% |

As n grows, the n^2 term dominates completely. The $1000n$ term becomes negligible.

****Real-World Analogy:****

Planning a cross-country trip (3000 miles):

- Driving: 3000 miles ($O(n)$)
- Walking: 3000 miles ($O(n)$ - same class!)
- But walking takes $100\times$ longer (constant factor)

However:

- Driving 3000 miles vs Driving 3000 miles stopping at every house ($O(n^2)$ - now checking every house along route)
- No amount of walking speed improvement beats driving for long distances

****Key Insight:**** Asymptotic analysis tells you if you need a car ($O(n)$) or if walking (also $O(n)$) is fine, or if you're doing something fundamentally slower like stopping at every house ($O(n^2)$).

****Question 43:**** You are explaining algorithm complexity to a non-technical manager. Write a simple explanation of Big-O notation using real-world analogies. Include $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, and $O(2^n)$.

****Answer:****

****Big-O Explained with Everyday Analogies****

**** $O(1)$ - Constant Time: The Light Switch****

- Flipping a light switch takes the same time whether you have 1 light or 100 lights in your house
- Operation time doesn't change with more stuff
- *Example: Looking up "Smith" in a phonebook by page number*

**** $O(\log n)$ - Logarithmic: Finding a Name in a Phonebook****

- To find "Johnson," open to middle, see you're in M's, discard half the book
- Each step cuts the problem in half
- Doubling the phonebook size adds ONLY ONE extra step
- *From 1000 to 2000 pages? Just one more split!*

**** $O(n)$ - Linear: Finding Your Keys****

- If you lost your keys, you might check every room once
- Twice as many rooms = twice as long to search
- Simple and straightforward - just more work with more stuff

****O(n log n) - Linearithmic: Organizing a Library****

- Sorting books alphabetically takes more than just looking at each book
- You need to compare and rearrange
- Efficient sorting feels like this - not too painful even with many books

****O(n²) - Quadratic: A Bad Wedding Seating Plan****

- If every guest must meet every other guest
- 10 guests = 45 introductions
- 100 guests = 4,950 introductions (100× guests = 100× more work!)
- This gets out of hand FAST

****O(2ⁿ) - Exponential: The Rice and Chessboard Problem****

- Put 1 grain on first square, double each next square
- By square 64, you'd need more rice than exists on Earth
- Add just ONE more square? Double the already-impossible amount!
- *Some problems literally cannot be solved for large inputs*

****Why This Matters for Business:****

| Complexity | For 100 items | For 1000 items | For 10,000 items |
|-----------------|---------------|-------------------|----------------------|
| --- --- --- --- | | | |
| O(1) | 1 min | 1 min | 1 min |
| O(log n) | 2 min | 3 min | 4 min |
| O(n) | 100 min | 1000 min (17 hrs) | 10,000 min (1 week) |
| O(n log n) | 200 min | 3000 min (2 days) | 40,000 min (1 month) |

| $O(n^2)$ | 10,000 min (1 week) | 1,000,000 min (2 years) | 100,000,000 min (190 years) |

| $O(2^n)$ | 10^{28} years | Universe ends | Never finishes |

****The Bottom Line:****

Choosing the right algorithm is like choosing the right tool. A faster computer helps $O(n)$ algorithms, but no computer can save an $O(2^n)$ algorithm for large problems. Pick your approach wisely!

SECTION E: CODE ANALYSIS QUESTIONS

****Question 44:**** Analyze the time complexity of this function:

```
```c
```

```
void mysteryFunction(int n) {
```

```
 int i = 1;
```

```
 while (i < n) {
```

```
 printf("%d ", i);
```

```
 i = i * 2;
```

```
 }
```

```
}
```

```
```
```

****Answer:****

****Step-by-step analysis:****

- i starts at 1
- Each iteration: i doubles (1, 2, 4, 8, 16, ...)
- Loop stops when $i \geq n$

****How many iterations?****

- After k iterations: $i = 2^k$
- Loop stops when $2^k \geq n$
- So $k \approx \log_2(n)$

****Example:****

- $n = 100$: i values = 1, 2, 4, 8, 16, 32, 64 (7 iterations)
- $\log_2(100) \approx 6.64$, so 7 iterations

****Time Complexity: $O(\log n)$ ****

****Why this matters:**** This is MUCH faster than $O(n)$. For $n = 1$ million, this loop runs only about 20 times!

****Question 45:**** Find the time complexity:

```
```c
```

```
void func(int n) {
 for (int i = 0; i < n; i++) {
 for (int j = i; j < n; j = j + 2) {
 printf("Operation\n");
 }
 }
}
```

```
```
```

****Answer:****

****Analysis:****

- Outer loop: i from 0 to n-1 (n iterations)
- Inner loop: j starts at i, increments by 2, stops at n

****Count operations for each i:****

- i = 0: j = 0, 2, 4, ... up to n → about $n/2$ iterations
- i = 1: j = 1, 3, 5, ... up to n → about $n/2$ iterations
- i = 2: j = 2, 4, 6, ... → about $(n-2)/2$ iterations
- Pattern continues...

****Total $\approx n \times (n/2)$ / something? Let's sum carefully:****

Better approach: Each pair (i,j) where $j \geq i$ and $j-i$ is even

For large n, this is approximately half of all pairs

Total pairs $\approx (n^2/2)/2 = n^2/4$

****Time Complexity: $O(n^2)$ ****

The $\div 2$ factor (step size) is just a constant - doesn't change the quadratic nature.

****Question 46:**** Compare these two functions. Which is more efficient for large n ?

```c

// Function A

```
void sumA(int n) {
 int total = 0;
 for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
 total += i * j;
}
```

// Function B

```
void sumB(int n) {
 int total = 0;
 for (int i = 0; i < n; i++)
 total += i * (n-1) * n / 2; // Using formula
```

}

...

**\*\*Answer:\*\***

**\*\*Function A analysis:\*\***

- Nested loops:  $n \times n = n^2$  iterations
- Each iteration: constant work
- Time complexity: **\*\* $O(n^2)$ \*\***

**\*\*Function B analysis:\*\***

- Single loop:  $n$  iterations
- Each iteration: constant work (using math formula)
- Time complexity: **\*\* $O(n)$ \*\***

**\*\*Comparison for different  $n$ :\*\***

| $n$ | Function A ( $n^2$ ops) | Function B ( $n$ ops) | Ratio |
|-----|-------------------------|-----------------------|-------|
|-----|-------------------------|-----------------------|-------|

| --- | --- | --- | --- |
|-----|-----|-----|-----|
|-----|-----|-----|-----|

|    |     |    |            |
|----|-----|----|------------|
| 10 | 100 | 10 | 10× faster |
|----|-----|----|------------|

|     |        |     |             |
|-----|--------|-----|-------------|
| 100 | 10,000 | 100 | 100× faster |
|-----|--------|-----|-------------|

|      |           |      |              |
|------|-----------|------|--------------|
| 1000 | 1,000,000 | 1000 | 1000× faster |
|------|-----------|------|--------------|

|           |           |           |                   |
|-----------|-----------|-----------|-------------------|
| 1,000,000 | $10^{12}$ | 1,000,000 | 1,000,000× faster |
|-----------|-----------|-----------|-------------------|

**\*\*Conclusion:\*\*** Function B is dramatically more efficient for large n. This shows the power of mathematical insight - recognizing that  $\sum(i \times j)$  over all pairs can be simplified, avoiding the nested loops entirely.

---

**\*\*Question 47:\*\*** What is the complexity of this recursive function?

```c

```
int fibonacci(int n) {  
    if (n <= 1)  
        return n;  
    return fibonacci(n-1) + fibonacci(n-2);  
}  
```
```

**\*\*Answer:\*\***

**\*\*Analysis:\*\***

This recursive Fibonacci makes two recursive calls for each call (except base cases).

**\*\*Call tree:\*\***

- fib(5) calls fib(4) and fib(3)
- fib(4) calls fib(3) and fib(2)
- fib(3) calls fib(2) and fib(1)

- And so on...

**\*\*Count of calls:\*\***

- Level 0: 1 call (fib(n))
- Level 1: 2 calls
- Level 2: 4 calls
- Level k:  $2^k$  calls
- Total calls  $\approx 2^n$  (actually about  $1.6^n$ , but still exponential)

**\*\*Time Complexity:  $O(2^n)$ \*\* - Exponential!**

**\*\*Why this is terrible:\*\***

- fib(40)  $\approx$  331,160,281 calls!
- fib(50) would take years

**\*\*Much better approach:\*\*** Use iteration or memoization

```c

```
int fibonacci_iterative(int n) {  
    int a = 0, b = 1, c;  
    for (int i = 2; i <= n; i++) {  
        c = a + b;  
        a = b;  
        b = c;  
    }  
    return b;  
}
```

```

This is  $O(n)$  - linear!

---

**\*\*Question 48:\*\*** Find the complexity:

```c

```
void complexFunction(int n) {  
    // Section 1  
    for (int i = 0; i < 1000; i++) {  
        printf("Setup\n");  
    }  
  
    // Section 2  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            printf("Nested\n");  
        }  
    }  
  
    // Section 3  
    for (int i = 0; i < n; i++) {  
        printf("Cleanup\n");  
    }  
}
```

...

****Answer:****

****Break down each section:****

****Section 1:**** Loop runs 1000 times (constant, independent of n)

- Complexity: $O(1)$

****Section 2:**** Nested loops, both running n times

- Complexity: $O(n^2)$

****Section 3:**** Single loop running n times

- Complexity: $O(n)$

****Total complexity = $O(1) + O(n^2) + O(n)$ ****

****Apply dominance rule:**** The highest-order term wins

- $O(n^2)$ dominates $O(n)$ and $O(1)$

****Final answer: $O(n^2)$ ****

****Important:**** Even though Section 1 runs 1000 operations (which might be more than Section 3's n operations for small n), for large n the n^2 term completely dominates. At $n = 1,000,000$:

- Section 1: 1000 ops (negligible)

- Section 2: 10^{12} ops (enormous)
- Section 3: 1,000,000 ops (tiny compared to Section 2)

SECTION F: MATCHING QUESTIONS

Question 49: Match each time complexity with its common name:

| Complexity | Name |
|------------------|-----------------|
| 1. $O(1)$ | A. Quadratic |
| 2. $O(\log n)$ | B. Linear |
| 3. $O(n)$ | C. Exponential |
| 4. $O(n \log n)$ | D. Constant |
| 5. $O(n^2)$ | E. Logarithmic |
| 6. $O(2^n)$ | F. Linearithmic |

Answers:

1 → D (Constant)

2 → E (Logarithmic)

3 → B (Linear)

4 → F (Linearithmic)

5 → A (Quadratic)

6 → C (Exponential)

Question 50: Match each algorithm with its typical time complexity:

| Algorithm | Complexity |
|--------------------------------|-------------------|
| --- | --- |
| 1. Binary search | A. $O(n^2)$ |
| 2. Linear search | B. $O(\log n)$ |
| 3. Bubble sort (worst case) | C. $O(1)$ |
| 4. Merge sort | D. $O(n)$ |
| 5. Array access by index | E. $O(n \log n)$ |
| 6. Fibonacci (naive recursive) | F. $O(2^n)$ |

Answers:

1 → B (Binary search = $O(\log n)$)
2 → D (Linear search = $O(n)$)
3 → A (Bubble sort worst = $O(n^2)$)
4 → E (Merge sort = $O(n \log n)$)
5 → C (Array index access = $O(1)$)
6 → F (Naive Fibonacci = $O(2^n)$)

SECTION G: FILL IN THE BLANKS

****Question 51:**** The three scenarios for algorithm performance are _____, _____, and _____.

****Answer:**** best case, average case, worst case

****Question 52:**** _____ notation describes the upper bound, _____ describes the lower bound, and _____ describes the tight bound.

****Answer:**** Big-O, Big-Omega, Big-Theta

****Question 53:**** When simplifying to Big-O, we drop _____ and _____.

****Answer:**** constants, lower-order terms

****Question 54:**** An algorithm is _____ when its upper bound matches the problem's lower bound.

****Answer:**** optimal

****Question 55:**** The time complexity of binary search is _____.

****Answer:**** $O(\log n)$

****Question 56:**** For comparison-based sorting, the proven lower bound is _____.

****Answer:**** $\Omega(n \log n)$

****Question 57:**** The four strategies for speeding up programs, in order of importance, are: _____, _____, _____, and _____.

****Answer:**** choose better algorithm, remove redundant computation, select better data structures, system-level optimizations

Question 58: Bubble sort's best case is _____, average case is _____, and worst case is _____.

Answer: $O(n)$, $O(n^2)$